

CST458 - Assignment 2

Name: Vishnu Dinesh

Roll No: IDK22CS065

Course: CST458 - Software Testing

Topic: Review of Testing Practices Followed in the Final Year Project

Part A - Project Details

1. Project Identification

- Project Title: GramaGIS - A Smart Panchayat GIS Portal
- Team Members: Akshay Kumar A S, Malavika Sasi, Sajin Pathrose, Vishnu Dinesh
- Guide Name: Dr. Dhanya S Pankaj
- Problem Addressed: Fragmented rural administrative and infrastructure data lacked a single spatial platform for transparent governance
- Technologies: HTML, CSS, JavaScript, Leaflet, Node.js, Express.js, PostgreSQL, PostGIS, GeoServer, JWT, Gemini API
- Modules: public portal, interactive map, smart query, feedback module, admin login, GIS data management, feedback review

Part B - Testing Practices

2. Testing Planning

- No formal test plan prepared
- No documented test case sheet available
- Requirements reviewed using objectives, workflows, and module list
- Expected outputs identified for login, feedback, search, and admin actions
- No separate test dataset prepared

3. Unit Testing

- Individually checked auth, feedback, query, and proxy routes
- Frontend logic checked for login, search, map, feedback form, and admin edit actions
- Unit testing done manually by team members during development
- White box ideas used informally while checking validations and branches
- No formal unit test cases prepared
- No unit testing framework used
- Defects found: validation issues, session expiry issues, schema mismatch cases, UI text issues

4. Integration Testing

- Frontend and Express backend integrated first
- JWT login flow tested from login page to protected admin page
- Feedback form tested with backend validation and database storage
- Smart query tested with frontend, Gemini, GeoServer, and displayed results
- Admin edit flow tested with schema fetch, WFS data load, and WFS-T update
- Module interactions tested manually through browser execution
- Issues found: CORS problems, timeout cases, schema mismatch, token passing errors

5. System Testing

- Full system tested as a complete working application

- Features tested: map loading, layer toggle, feature info, smart query, feedback, login, admin edit
- Citizen workflow and admin workflow checked end-to-end
- UI, backend, database, and output consistency checked together
- Issues found: external service failure cases, empty-result handling, auth interruptions

6. Acceptance Testing

- Guide and faculty reviewed the project during demos and evaluations
- Feedback received on interface clarity, feature usefulness, and correctness of outputs
- Suggested corrections were implemented in UI and workflow behavior
- Main project objective was achieved
- Formal testing by real Panchayat users was limited

7. Functional Testing

- Major functions tested against project requirements
- Tested functions: login, protected access, layer loading, search, feedback submit, feedback update, GIS edit, download
- Both valid and invalid inputs were tried
- Missing tests: malformed AI output, concurrent edits, unsupported geometry, external failure cases
- Issues found: search parsing failure, invalid input handling gaps, GeoServer field mismatch

8. Performance / Stress Testing

- Response time observed informally during application use
- Login, search, WMS/WFS loading, feedback submission, and admin save delay were checked
- Timeout handling exists in backend service calls
- No formal load testing done
- No multiple-user stress testing done
- No CPU or memory profiling report prepared
- Performance testing remained limited

9. Usability Testing

- Ease of use checked manually for landing page, map, feedback page, login, and admin dashboard
- Navigation and instruction clarity were checked during usage
- Search help text, alerts, and status labels were improved during development
- Some UI changes were made based on observed difficulties
- Issues noticed: GIS-specific terms, complex admin flow, limited mobile verification

10. Regression Testing

- Previous features rechecked manually after modifications
- Repeated testing done after updates in auth, feedback, query, and proxy modules
- Bug fixes could introduce new errors because modules are tightly connected
- No regression checklist maintained
- No automated regression suite available

Part C - Testing Approaches Used

11. Black Box Testing Practices

- Input-output testing done for login, feedback, search, map data load, and downloads
- Valid and invalid credentials, categories, wards, layers, and statuses were tried
- Boundary checks considered for longitude, latitude, ward parsing, and login attempt limits
- Invalid conditions tested using empty fields, wrong token, incomplete coordinates, and unsupported actions
- Examples: invalid login rejection, invalid feedback coordinates rejection, unsupported layer rejection

12. White Box Testing Practices

- Code-level logic checked manually while debugging
- Conditions, branches, and loops checked in auth, feedback validation, role checks, and proxy logic
- Internal paths checked for timeout, invalid token, missing env, and invalid GeoServer response cases
- Exception handling paths were also verified informally
- Examples: login rate-limit branch, status normalization logic, WKT-to-GML conversion path

13. Grey Box Testing Practices

- Testing done with partial internal knowledge of routes, schema, database behavior, and roles
- API structure and database knowledge helped identify errors faster
- Both UI behavior and internal workflow were used during testing
- Examples: checking UI search against backend query behavior, checking feedback status persistence, checking schema-driven admin editor

Part D - Evaluation and Improvements

14. Gaps in Existing Testing Process

- No formal test planning
- No documented test cases
- No automated unit testing
- No automated integration or regression testing
- Weak performance and stress testing
- Limited acceptance testing with real users
- Missing coverage for some edge cases and external failures
- No defect tracking tool used
- Time and coordination constraints reduced testing depth

15. Suggested Improvements

- Add unit tests for validation logic, auth, query parsing, and XML generation
- Add integration tests for login, feedback, query, and admin CRUD workflows
- Prepare a system test checklist before release or demo
- Perform formal acceptance testing with guide and real target users
- Maintain a regression checklist after every major update
- Document test cases with valid, invalid, and boundary inputs
- Use automation tools such as Jest, Supertest, and Playwright
- Improve usability and performance testing with structured observation and load tools
- Use proper defect tracking and retesting records

16. Final Reflection

- Manual functional, integration, and system testing were done reasonably well
- Formal planning, documentation, automation, and load testing were insufficient
- Test design and acceptance planning should have started earlier
- Better testing would improve reliability, usability, and overall project quality